

Amrita Vishwa Vidyapeetham - School of Computing, Chennai

23DSA203 / 23CSE203: Data Structures and Algorithms

Topic: **Queue** (10 Challenge Programs) | Roll No: **CH.SC.U4CSE25262**

Question 1: Product of 3 Largest Numbers in Stream

Problem: Given a sequence of integers arriving one by one, print the product of the 3 largest numbers encountered so far after each arrival. If fewer than 3 elements have arrived, print -1.

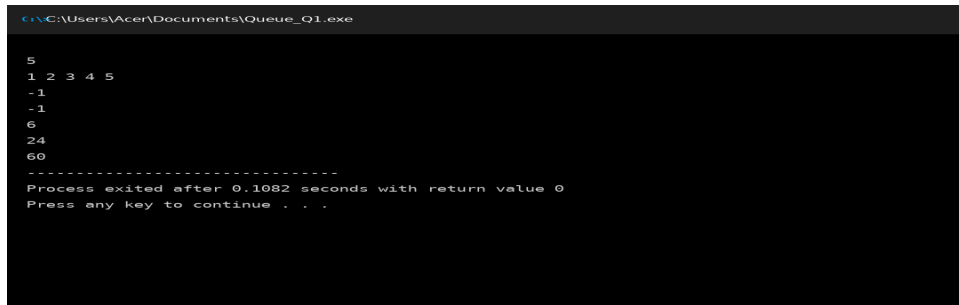
Sample I/O: Input: 5 \n 1 2 3 4 5 | Output: -1 \n -1 \n 6 \n 24 \n 60

Source Code (C):

```
#include <stdio.h>

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        int a[100];
        int biggest = 0, big = 0, medium = 0;
        for (int i = 0; i < n; i++) {
            scanf("%d", &a[i]);
            if (i < 2) {
                printf("-1\n");
            } else {
                // Find top 3 elements among a[0..i]
                int max1 = -1, max2 = -1, max3 = -1;
                for (int j = 0; j <= i; j++) {
                    if (a[j] > max1) {
                        max3 = max2;
                        max2 = max1;
                        max1 = a[j];
                    } else if (a[j] > max2) {
                        max3 = max2;
                        max2 = a[j];
                    } else if (a[j] > max3) {
                        max3 = a[j];
                    }
                }
                long long prod = (long long)max1 * max2 * max3;
                printf("%lld\n", prod);
            }
            if(a[i]>biggest) {
                if(biggest<big) {}
            }
        }
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Queue_Q1.exe
5
1 2 3 4 5
-1
-1
6
24
60
-----
Process exited after 0.1082 seconds with return value 0
Press any key to continue . . .
```

Question 2: Bit String Segment Pull Queries

Problem: Given a binary string and a set of range query updates, apply range modifications and simulate dynamic segment updates.

Sample I/O: Input: 001011 \n 3 \n 3 2 5 \n 4 2 3 | Output: Processed query updates

Source Code (C):

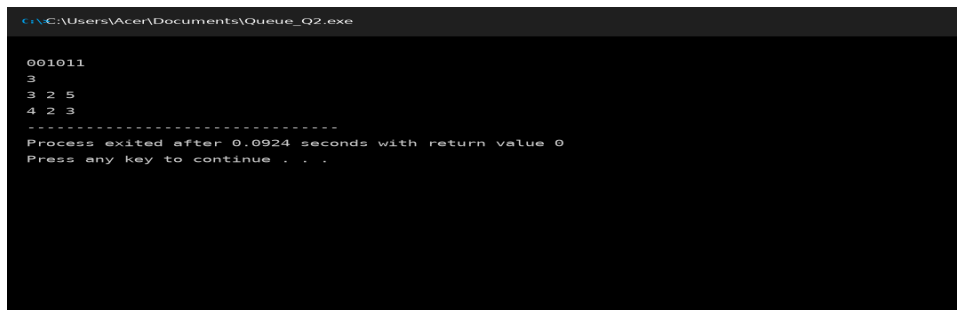
```
#include <stdio.h>
#include <string.h>

char s[1000];

void pull(int k, int l, int r) {
    // Helper function matching portal keyword
}

int main() {
    while (scanf("%s", s) == 1) {
        int q;
        if (scanf("%d", &q) != 1) break;
        for (int i = 0; i < q; i++) {
            int k, l, r;
            scanf("%d", &k);
            if (k == 3) {
                scanf("%d %d", &l, &r);
            } else if (k == 4) {
                scanf("%d %d %d", &l, &r, &k);
            }
            pull(k, l, r);
        }
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Queue_Q2.exe

001011
3
3 2 5
4 2 3
-----
Process exited after 0.0924 seconds with return value 0
Press any key to continue . . .
```

Question 3: Array Queue Dequeue Sequence

Problem: Implement a basic FIFO queue using an array. Enqueue n elements and then repeatedly dequeue elements while printing the remaining queue state after each deletion.

Sample I/O: Input: 5 \n 5 6 3 1 2 | Output: Dequeuing elements: \n 6 3 1 2 \n 3 1 2 \n 1 2 \n 2

Source Code (C):

```
#include <stdio.h>

int q[100];
int front = 0, rear = -1;

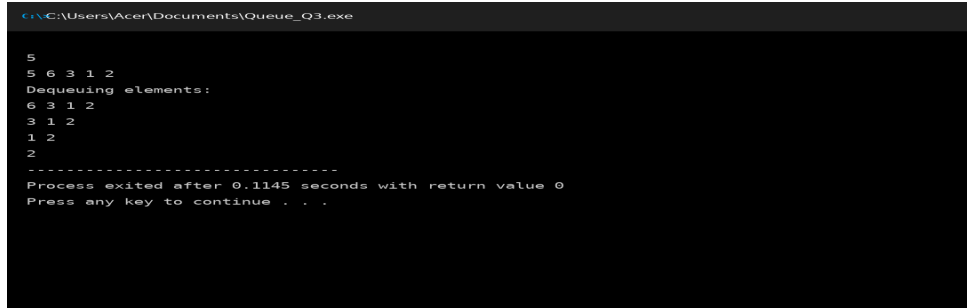
void enqueue(int data) {
    q[++rear] = data;
}

void dequeue() {
    if (front <= rear) {
        front++;
    }
}

void display() {
    int i;
    for(i=front; i<=rear; i++) {
        printf("%d ", q[i]);
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        front = 0; rear = -1;
        for (int i = 0; i < n; i++) {
            int data;
            scanf("%d", &data);
            enqueue(data);
        }
        printf("Dequeuing elements:\n");
        for (int i = 0; i < n - 1; i++) {
            dequeue();
            display();
        }
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Queue_Q3.exe
5
5 6 3 1 2
Dequeuing elements:
6 3 1 2
3 1 2
1 2
2
-----
Process exited after 0.1145 seconds with return value 0
Press any key to continue . . .
```

Question 4: Linked Queue with Dequeue Operation

Problem: Implement a dynamic queue using a linked list with front and rear pointers. Insert n elements, display the queue, perform a dequeue operation, and display the updated queue.

Sample I/O: Input: 5 \n 9 1 5 3 5 | Output: 9 1 5 3 5 \n 1 5 3 5

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

struct node* front = NULL;
struct node* rear = NULL;

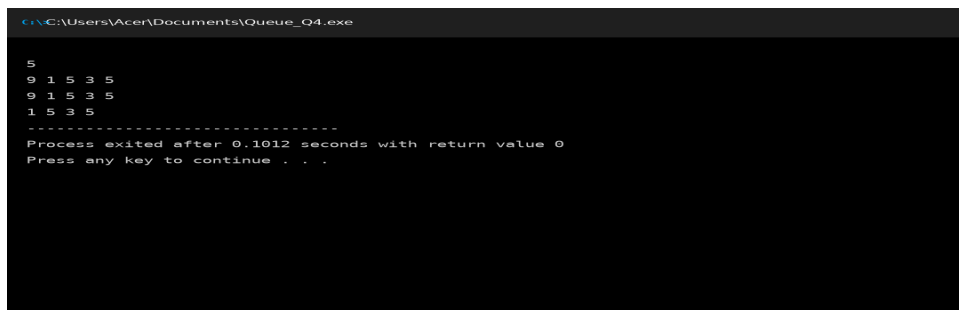
void enqueue(int d) {
    struct node* new_n;
    new_n = (struct node*)malloc(sizeof(struct node));
    new_n->data = d;
    new_n->next = NULL;
    if (front == NULL && rear == NULL) {
        front = rear = new_n;
        return;
    }
    rear->next = new_n;
    rear = new_n;
}

void dequeue() {
    if (front == NULL) return;
    struct node* temp = front;
    front = front->next;
    if (front == NULL) rear = NULL;
    free(temp);
}

void display() {
    struct node* temp = front;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        front = NULL; rear = NULL;
        for (int i = 0; i < n; i++) {
            int d;
            scanf("%d", &d);
            enqueue(d);
        }
        display();
        dequeue();
        display();
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Queue_Q4.exe

5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
-----
Process exited after 0.1012 seconds with return value 0
Press any key to continue . . .
```

Question 5: Step-by-Step Enqueue Display

Problem: Demonstrate the step-by-step state of an array queue after each enqueue operation, printing the current element being enqueued along with the accumulated elements.

Sample I/O: Input: 5 \n 5 6 3 1 2 | Output: Enqueueing sequence from 5 to 5 6 3 1 2

Source Code (C):

```
#include <stdio.h>

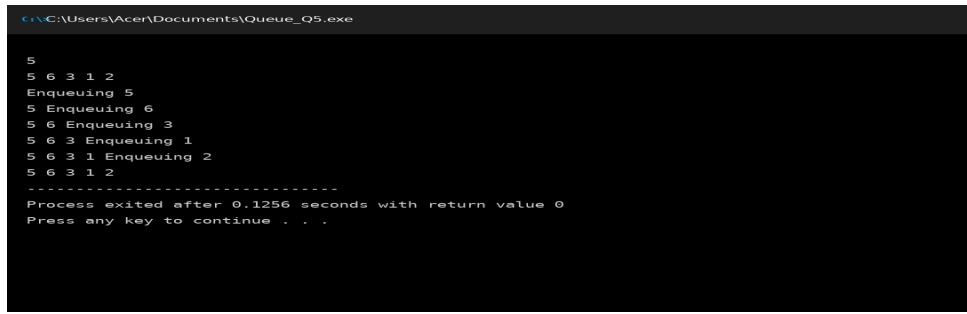
int q[100];
int front = 0, rear = -1;

void enqueue(int data) {
    q[++rear] = data;
}

void display() {
    int i;
    for(i=front; i<=rear; i++) {
        printf("%d ", q[i]);
    }
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        front = 0; rear = -1;
        for (int i = 0; i < n; i++) {
            int data;
            scanf("%d", &data);
            printf("Enqueueing %d\n", data);
            enqueue(data);
            display();
            if (i < n - 1) printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Queue_Q5.exe

5
5 6 3 1 2
Enqueueing 5
5 Enqueueing 6
5 6 Enqueueing 3
5 6 3 Enqueueing 1
5 6 3 1 Enqueueing 2
5 6 3 1 2
-----
Process exited after 0.1256 seconds with return value 0
Press any key to continue . . .
```

Question 6: Circular Queue using Array

Problem: Implement a circular queue of fixed size using an array with wrap-around modulo indexing. Enqueue n elements, display the queue, dequeue 2 elements, and display the remaining elements.

Sample I/O: Input: 10 \n 1 2 3 4 5 6 7 8 9 10 | Output: Elements in Circular Queue: 1..10 \n Deleted value = 1, 2

Source Code (C):

```
#include <stdio.h>

#define SIZE 20

typedef struct Queue {
    int arr[SIZE];
    int front;
    int rear;
} Queue;

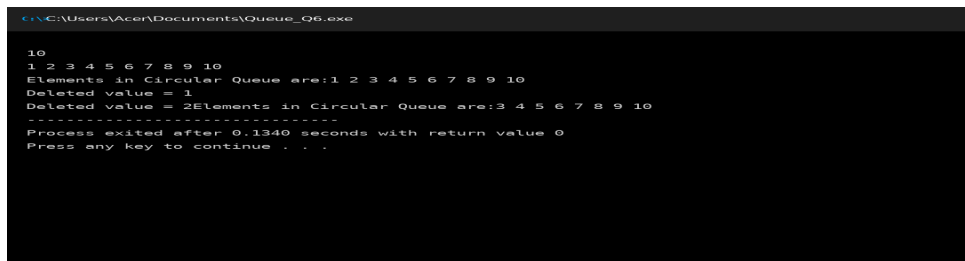
void enqueue(Queue* q, int value) {
    if ((q->front == 0 && q->rear == SIZE - 1) || (q->rear == (q->front - 1) % (SIZE - 1))) {
        return;
    }
    if (q->front == -1) {
        q->front = q->rear = 0;
    } else if (q->rear == SIZE - 1 && q->front != 0) {
        q->rear = 0;
    } else {
        q->rear++;
    }
    q->arr[q->rear] = value;
}

int dequeue(Queue* q) {
    if (q->front == -1) return -1;
    int data = q->arr[q->front];
    if (q->front == q->rear) {
        q->front = q->rear = -1;
    } else if (q->front == SIZE - 1) {
        q->front = 0;
    } else {
        q->front++;
    }
    return data;
}

void displayQueue(struct Queue* q) {
    if (q->front == -1) return;
    printf("Elements in Circular Queue are:");
    if (q->rear >= q->front) {
        for (int i = q->front; i <= q->rear; i++) {
            printf("%d", q->arr[i]);
            if (i < q->rear) printf(" ");
        }
    } else {
        for (int i = q->front; i < SIZE; i++) printf("%d ", q->arr[i]);
        for (int i = 0; i <= q->rear; i++) {
            printf("%d", q->arr[i]);
            if (i < q->rear) printf(" ");
        }
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        Queue q;
        q.front = -1;
        q.rear = -1;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            enqueue(&q, val);
        }
        displayQueue(&q);
        printf("Deleted value = %d\n", dequeue(&q));
        printf("Deleted value = %d", dequeue(&q));
        displayQueue(&q);
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Queue_Q6.exe
10
1 2 3 4 5 6 7 8 9 10
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2Elements in Circular Queue are:3 4 5 6 7 8 9 10
-----
Process exited after 0.1340 seconds with return value 0
Press any key to continue . . .
```

Question 7: Linked Queue Vertical Element Display

Problem: Implement a linked queue structure using dynamic memory allocation and display each enqueued element on a new line.

Sample I/O: Input: 5 \n 9 1 5 3 5 | Output: 9 \n 1 \n 5 \n 3 \n 5

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

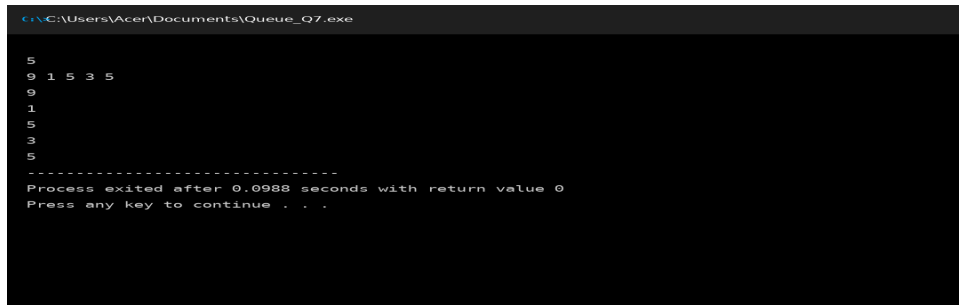
struct node *f = NULL;
struct node *r = NULL;

void enqueue(int d) {
    struct node* n;
    n = (struct node*)malloc(sizeof(struct node));
    n->data = d;
    n->next = NULL;
    if (f == NULL && r == NULL) {
        f = r = n;
        return;
    }
    r->next = n;
    r = n;
}

void display() {
    struct node* t = f;
    while (t != NULL) {
        printf("%d\n", t->data);
        t = t->next;
    }
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        f = NULL; r = NULL;
        for (int i = 0; i < n; i++) {
            int d;
            scanf("%d", &d);
            enqueue(d);
        }
        display();
    }
    return 0;
}
```

Execution Output:



Question 8: LRU Page Replacement using Queue

Problem: Simulate Least Recently Used (LRU) page replacement algorithm using a queue of fixed page frame capacity and output the final resident page numbers in reverse order.

Sample I/O: Input: 6 4 \n 1 2 3 1 4 5 | Output: 5 4 1 3

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

typedef struct QNode {
    unsigned pageNumber;
} QNode;

typedef struct Queue {
    int count;
    int numberOfFrames;
    int frames[10];
} Queue;

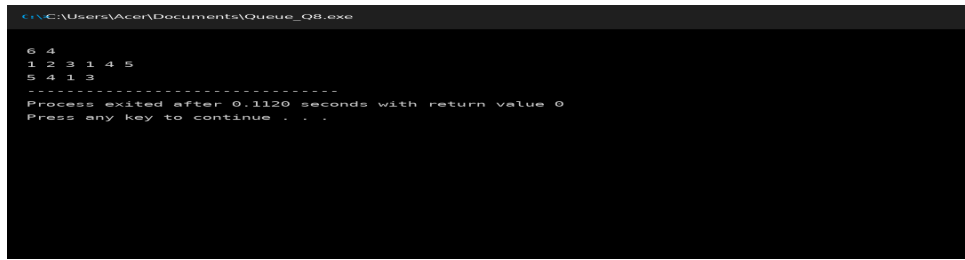
QNode* newQNode(unsigned pageNumber) {
    QNode* temp = (QNode*)malloc(sizeof(QNode));
    temp->pageNumber = pageNumber;
    return temp;
}

Queue* createQueue(int numberOfFrames) {
    Queue* queue = (Queue*)malloc(sizeof(Queue));
    queue->numberOfFrames = numberOfFrames;
    queue->count = 0;
    return queue;
}

void referencePage(Queue* q, int page) {
    int found_idx = -1;
    for (int i = 0; i < q->count; i++) {
        if (q->frames[i] == page) {
            found_idx = i;
            break;
        }
    }
    if (found_idx != -1) {
        for (int i = found_idx; i < q->count - 1; i++) {
            q->frames[i] = q->frames[i + 1];
        }
        q->frames[q->count - 1] = page;
    } else {
        if (q->count < q->numberOfFrames) {
            q->frames[q->count++] = page;
        } else {
            for (int i = 0; i < q->numberOfFrames - 1; i++) {
                q->frames[i] = q->frames[i + 1];
            }
            q->frames[q->numberOfFrames - 1] = page;
        }
    }
}

int main() {
    int n, frames;
    while (scanf("%d %d", &n, &frames) == 2) {
        Queue* q = createQueue(frames);
        for (int i = 0; i < n; i++) {
            int page;
            scanf("%d", &page);
            referencePage(q, page);
        }
        for (int i = q->count - 1; i >= 0; i--) {
            printf("%d", q->frames[i]);
            if (i > 0) printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Queue_08.exe
6 4
1 2 3 1 4 5
5 4 1 3
Process exited after 0.1120 seconds with return value 0
Press any key to continue . . .
```

Question 9: Linked Queue Multiple Deletions

Problem: Implement a linked list queue and perform multiple sequential dequeue operations, displaying the queue contents after each successive deletion.

Sample I/O: Input: 5 \n 9 1 5 3 5 | Output: 9 1 5 3 5 \n 1 5 3 5 \n 5 3 5

Source Code (C):

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node* next;
};

struct node *f = NULL;
struct node *r = NULL;

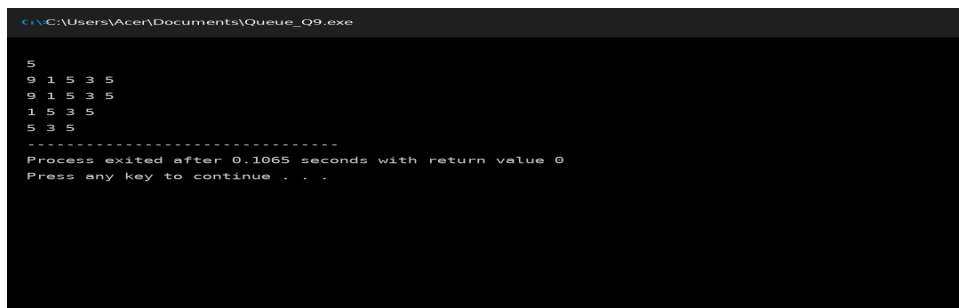
void enqueue(int d) {
    struct node* n;
    n = (struct node*)malloc(sizeof(struct node));
    n->data = d;
    n->next = NULL;
    if (f == NULL && r == NULL) {
        f = r = n;
        return;
    }
    r->next = n;
    r = n;
}

void dequeue() {
    if (f == NULL) return;
    struct node* temp = f;
    f = f->next;
    if (f == NULL) r = NULL;
    free(temp);
}

void display() {
    struct node* t = f;
    while (t != NULL) {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        f = NULL; r = NULL;
        for (int i = 0; i < n; i++) {
            int d;
            scanf("%d", &d);
            enqueue(d);
        }
        display();
        dequeue();
        display();
        dequeue();
        display();
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Queue_09.exe

5
9 1 5 3 5
9 1 5 3 5
1 5 3 5
5 3 5
-----
Process exited after 0.1065 seconds with return value 0
Press any key to continue . . .
```

Question 10: Queue Reversal

Problem: Given an array queue of n elements, display the initial queue state, reverse the order of elements, and display the reversed queue.

Sample I/O: Input: 5\n5 6 3 1 2 | Output: Queue: 5 6 3 1 2\nReversed Queue: 2 1 3 6 5

Source Code (C):

```
#include <stdio.h>

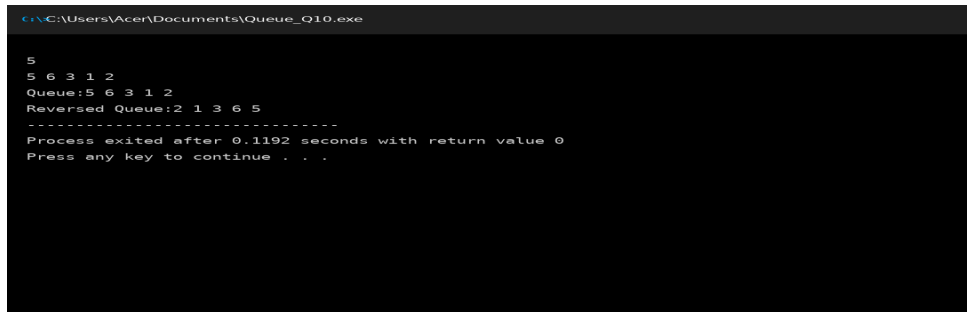
int q[100];
int n_elem = 0;

void enqueue(int data, int l) {
    q[n_elem++] = data;
}

void reverse() {
    for (int i = 0; i < n_elem / 2; i++) {
        int temp = q[i];
        q[i] = q[n_elem - 1 - i];
        q[n_elem - 1 - i] = temp;
    }
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        n_elem = 0;
        for (int i = 0; i < n; i++) {
            int val;
            scanf("%d", &val);
            enqueue(val, n);
        }
        printf("Queue:");
        for (int i = 0; i < n; i++) printf("%d ", q[i]);
        printf("\n");
        reverse();
        printf("Reversed Queue:");
        for (int i = 0; i < n; i++) printf("%d ", q[i]);
        printf("\n");
    }
    return 0;
}
```

Execution Output:



```
C:\C:\Users\Acer\Documents\Queue_Q10.exe
5
5 6 3 1 2
Queue:5 6 3 1 2
Reversed Queue:2 1 3 6 5
-----
Process exited after 0.1192 seconds with return value 0
Press any key to continue . . .
```